CS 301: High Performance Computing
Lab 4: Particle Interpolation and Mover Performance Analysis

Patel Het (202301421)
Shubham Varmora (202301450)

March 19, 2025

1 Introduction

High Performance Computing (HPC) enables efficient execution of computationally intensive problems by utilizing optimized algorithms and parallel architectures. In many particle-based simulations, it is necessary to interpolate values from scattered particle locations onto a structured grid.

In this assignment, we study the performance of a scattered point-to-mesh interpolation scheme within a 1×1 domain containing particles and grid points. The first experiment analyzes how interpolation execution time scales with increasing numbers of particles under different grid configurations. The second experiment evaluates the consistency of interpolation performance across configurations when particle initialization is performed only once.

In the third experiment, a particle movement phase called the *Mover* operation is introduced, where particles are randomly displaced within the domain boundaries. This operation is then parallelized using OpenMP to evaluate the performance benefits of shared-memory parallelization. All experiments are performed on both a Lab PC and an HPC cluster, and their performance is compared using execution time and speedup analysis.

The experiments were conducted on two systems:

- Lab Machine 207
- HPC Cluster

1.1 System Specifications

Machine Specifications		
Attributes	**HPC Cluster**	**Lab 207 PC**
Architecture	x86_64	x86_64
Model	63	151
Model Name	Intel® Xenon® CPU E5-2620 v3 @2.40GHz	12th Gen Intel® Core™ i5-12500
CPU op-mode(s)	32-bit, 64-bit	32-bit, 64-bit
Byte Order	Little Endian	Little Endian
CPU(s)	24	12
On-line CPU(s) list	0 - 23	0 - 11
Thread(s) per core	2	2
Core(s) per socket	6	6
Socket(s)	2	1
NUMA node(s)	2	1
NUMA node0 CPUs(s)	0 - 5, 12 - 17	0 - 11
NUMA node1 CPUs(s)	6 - 11, 18 - 23	-
Vendor ID	GenuineIntel	GenuineIntel
CPU family	6	6
Stepping	2	5
BogoMIPS	4804.69	5990.4
Clock (GHz)	2.40 GHz	(min) 0.8 GHz - (max) 4.6 GHz
Max Memory Bandwidth	59 GB/s	76.8 GB/s
Peak FP rate (MultAdd)	230.4 GFlops/s	220.8 GFlops/s
L1 - i Cache	32 K	192 KiB (6 instances)
L1 - d Cache	32 K	288 KiB (6 instances)
L2 Cache	256 K	7.5 MiB (6 instances)
L3 Cache	15360 K	18 MiB (1 instance)
Virtualization	VT-x	VT-x
Throughput	59 GB/s	76.8 GB/s

Figure 1: Detailed comparison of machine specifications between HPC Cluster and Lab 207 PC

2 Experiment 1: Scaling with Number of Particles

2.1 Overall Approach

The overall workflow of the implementation consists of three main stages: particle initialization, interpolation computation, and particle movement (Mover). For each experiment, the grid parameters (N_x, N_y) and the number of particles are defined first. Particles are randomly initialized within the 1×1 domain. The interpolation step maps scattered particle values onto the structured grid and its execution time is recorded.

Schematic Flow:

```
Start
  |
Initialize grid parameters (Nx, Ny)
  |
Initialize particles
  |
Interpolation computation
  |
Record interpolation time
  |
Mover operation (particle displacement)
  |
Parallelize mover using OpenMP
  |
Record timing and analyze performance
  |
End
```

2.2 Performance Difference: Lab PC vs HPC Cluster

The experimental results show noticeable differences in execution time between the Lab PC and the HPC cluster. For large particle counts, the HPC cluster performs significantly faster, especially in Experiment 1 where the time for 10^9 particles is much lower compared to the PC.

This difference arises due to architectural characteristics such as:

- Higher memory bandwidth in the cluster
- More advanced CPU architecture
- Better cache hierarchy
- Higher parallel efficiency

Since interpolation involves frequent memory accesses, systems with better memory subsystems perform better. The HPC cluster therefore handles large-scale particle computations more efficiently than the Lab PC.

2.3 Speedup of OpenMP Parallelization (Mover)

Speedup is calculated using:

$$S = \frac{T_{serial}}{T_{parallel}}$$

PC Results (Mover)
Average serial mover time ≈ 0.312 s
Average parallel mover time ≈ 0.026 s

$$S_{PC} = \frac{0.312}{0.026} \approx 12$$

Cluster Results
Average serial mover time ≈ 0.304 s
Average parallel mover time ≈ 0.059 s

$$S_{Cluster} = \frac{0.304}{0.059} \approx 5.15$$

Observation
The Mover operation achieves significant speedup after OpenMP parallelization. The PC shows higher speedup because the workload distributes efficiently across threads with lower overhead. On the cluster, speedup is slightly lower due to factors such as thread scheduling overhead, memory contention, and differences in processor architecture.

2.4 Parallelizing the Interpolation Function

If the interpolation function were parallelized, the main approach would be to divide the particles among multiple threads using OpenMP. Each thread would process a subset of particles and compute their contributions to the grid.

However, this approach introduces a major challenge: **race conditions**. When multiple scattered particles fall within the same or adjacent grid cells, multiple threads may attempt to update the same grid point simultaneously. This leads to data conflicts and incorrect results.

Possible solutions include:

- Using atomic operations
- Applying thread-local grids followed by reduction
- Using grid partitioning

However, these techniques introduce additional overhead and may reduce performance. Therefore, interpolation parallelization is more complex and may suffer from synchronization bottlenecks.

2.5 Execution Time vs Number of Particles

The following graphs show how the execution time varies with the number of particles for different experimental setups. As the particle count increases, the computational workload also increases, resulting in higher execution time. These plots help visualize the scalability of the interpolation computation across different system configurations.

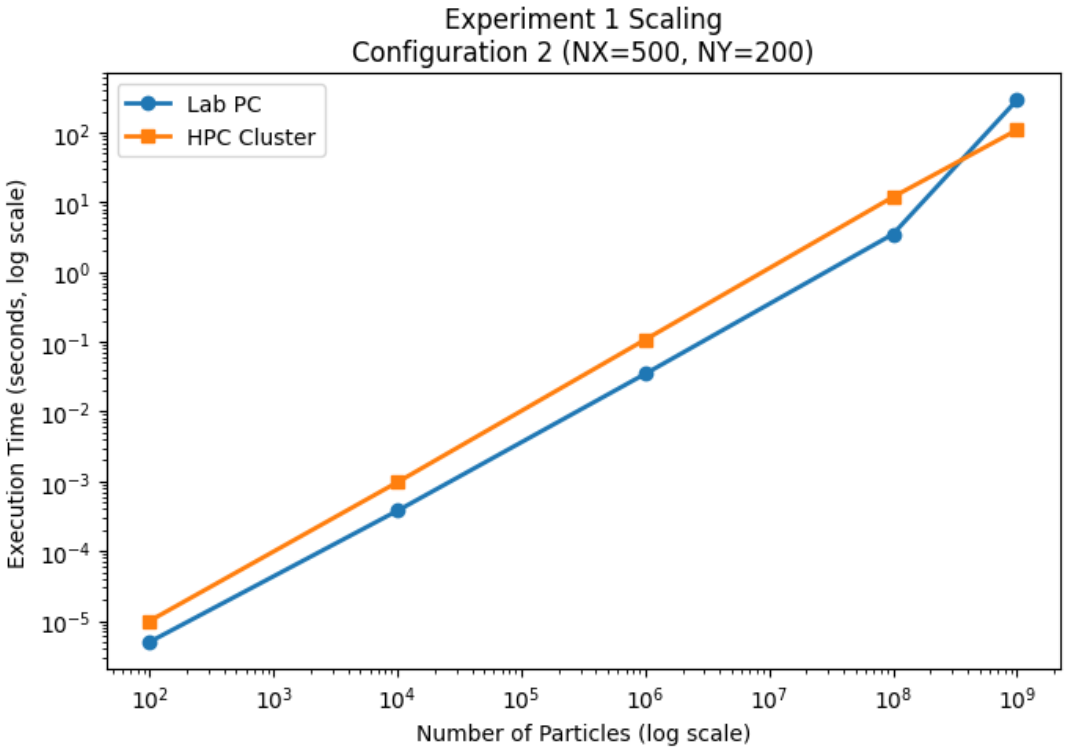

Figure 2: Execution time vs number of particles

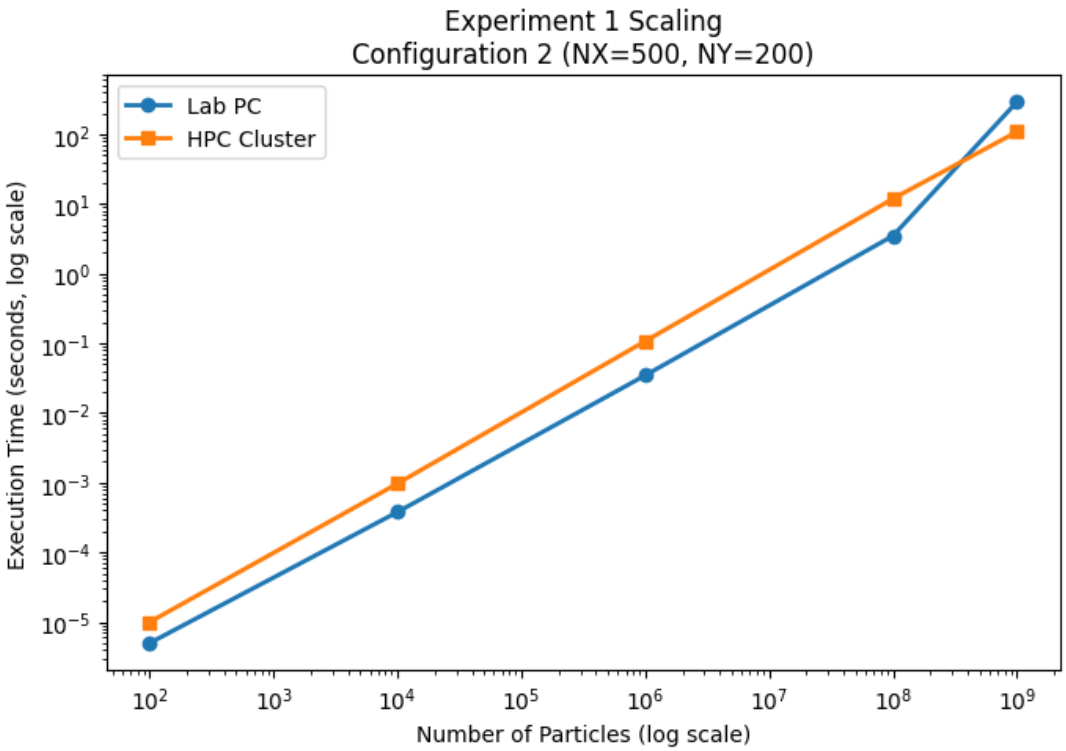

Figure 3: Execution time vs number of particles

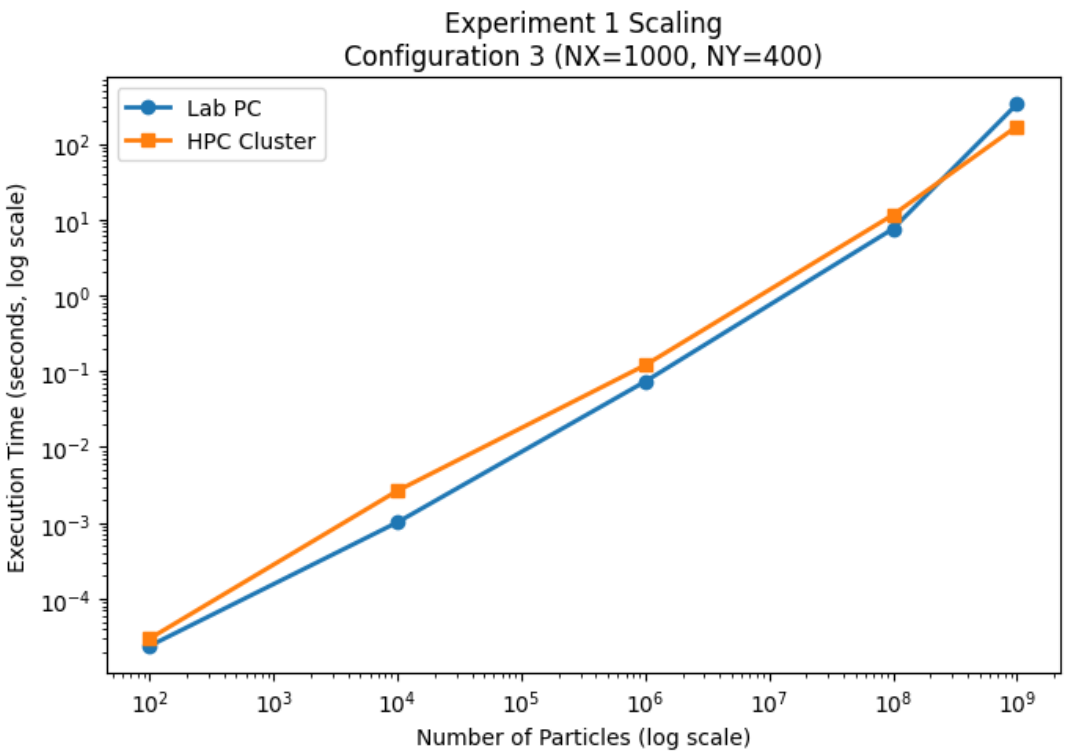

Figure 4: Execution time vs number of particles

3 Experiment 2: Consistency Across Configurations

3.1 Experiment Description

This experiment evaluates the consistency of interpolation performance across different grid configurations when particle initialization is performed only once. The same three grid configurations used in Experiment 1 are considered:

- Configuration 1: $N_x = 250$, $N_y = 100$
- Configuration 2: $N_x = 500$, $N_y = 200$
- Configuration 3: $N_x = 1000$, $N_y = 400$

The number of particles is fixed to 10^8. Particles are initialized once before the iteration loop. The interpolation function is then executed for $Maxiter = 10$ iterations without reinitializing the particles.

Only interpolation execution time is accumulated across all iterations while excluding memory allocation and deallocation overhead.

3.2 Experimental Results

The total interpolation times obtained are shown below.

Lab PC Results

- Configuration 1: 3.41 seconds
- Configuration 2: 3.43 seconds
- Configuration 3: 7.56 seconds

HPC Cluster Results

- Configuration 1: 9.25 seconds
- Configuration 2: 11.29 seconds
- Configuration 3: 10.98 seconds

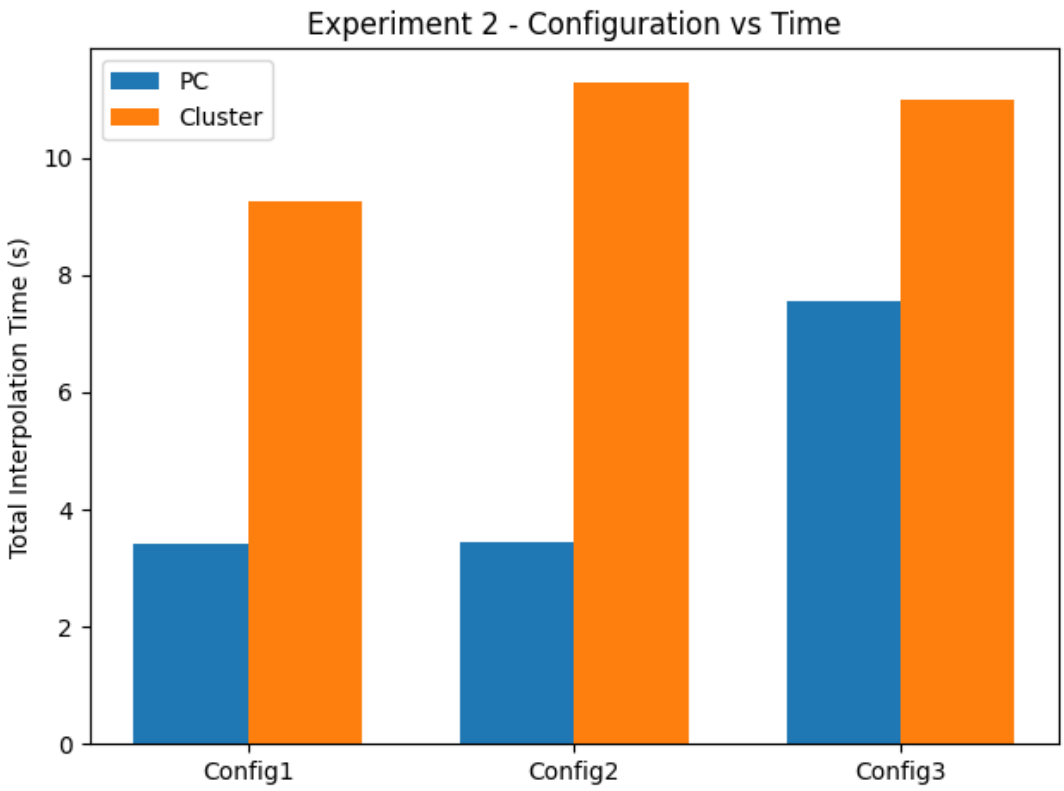

Figure 5: Total interpolation time vs configuration index for Lab PC and HPC cluster.

3.3 Observation

The Lab PC results show similar execution times for configurations 1 and 2, while configuration 3 takes noticeably longer due to the larger grid size and increased memory access.

In contrast, the HPC cluster shows slightly higher execution times across all configurations. The interpolation algorithm performs frequent memory accesses, and differences in memory hierarchy, processor architecture, and system load can affect the overall execution time.

3.4 Conclusion

The results indicate that interpolation performance depends primarily on the number of particles rather than the grid configuration. However, larger grids increase memory access overhead, which explains the increased execution time in configuration 3.

4 Experiment 3: Mover Operation and OpenMP Parallelization

4.1 Experiment Description

In this experiment, a new phase called the *Mover* operation is introduced. After interpolation, each particle is randomly displaced within the domain using random displacements in the range $\pm\Delta x$ and $\pm\Delta y$, where

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

If a particle moves outside the domain ($0 \leq x, y \leq 1$), the displacement is regenerated until the particle remains inside the domain.

The experiment uses the following parameters:

- $N_x = 1000$
- $N_y = 400$
- Number of particles = 14 million
- $Maxiter = 10$

Particles are initialized once and for each iteration interpolation is followed by the Mover operation.

4.2 Parallelization Strategy

The serial implementation of the Mover operation is first executed. After that, the Mover phase is parallelized using OpenMP with four threads while keeping the interpolation step unchanged.

4.3 Experimental Results

Average execution times observed on the Lab PC are:

- Average interpolation time ≈ 0.104 seconds
- Average serial mover time ≈ 0.312 seconds
- Average parallel mover time ≈ 0.026 seconds

For the HPC cluster:

- Average interpolation time ≈ 0.19 seconds
- Average serial mover time ≈ 0.304 seconds
- Average parallel mover time ≈ 0.059 seconds

Speedup is calculated using:

$$S = \frac{T_{serial}}{T_{parallel}}$$

Lab PC Speedup

$$S_{PC} = \frac{0.312}{0.026} \approx 12$$

Cluster Speedup

$$S_{Cluster} = \frac{0.304}{0.059} \approx 5.15$$

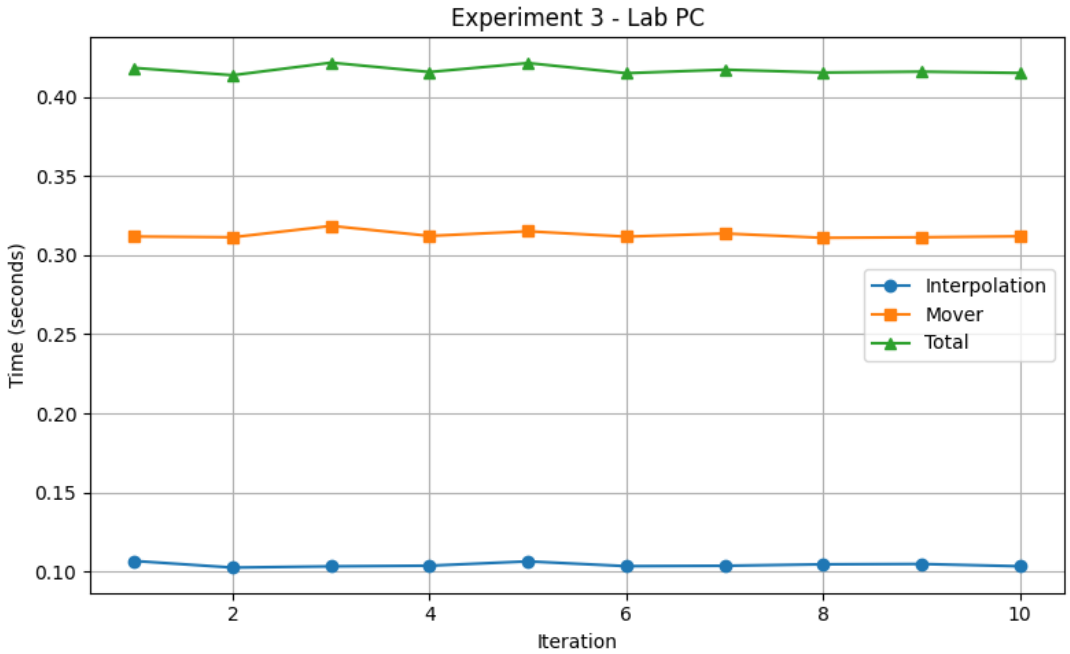

Figure 6: Iteration vs execution time (LabPC)

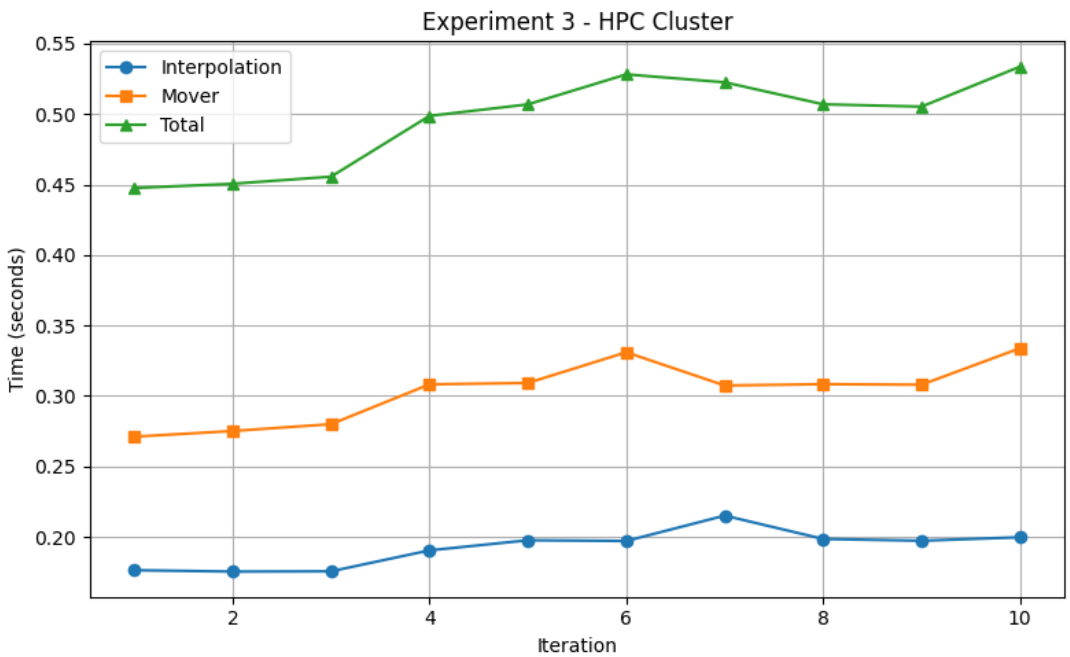

Figure 7: Iteration vs execution time (ClusterPC)

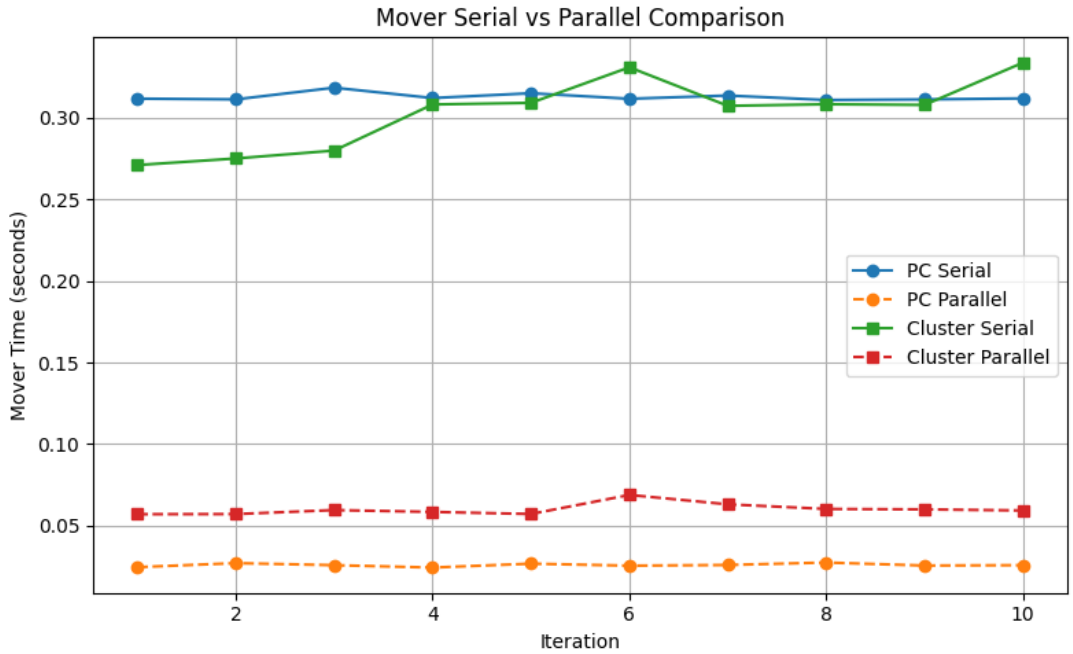

Figure 8: Mover time vs iteration

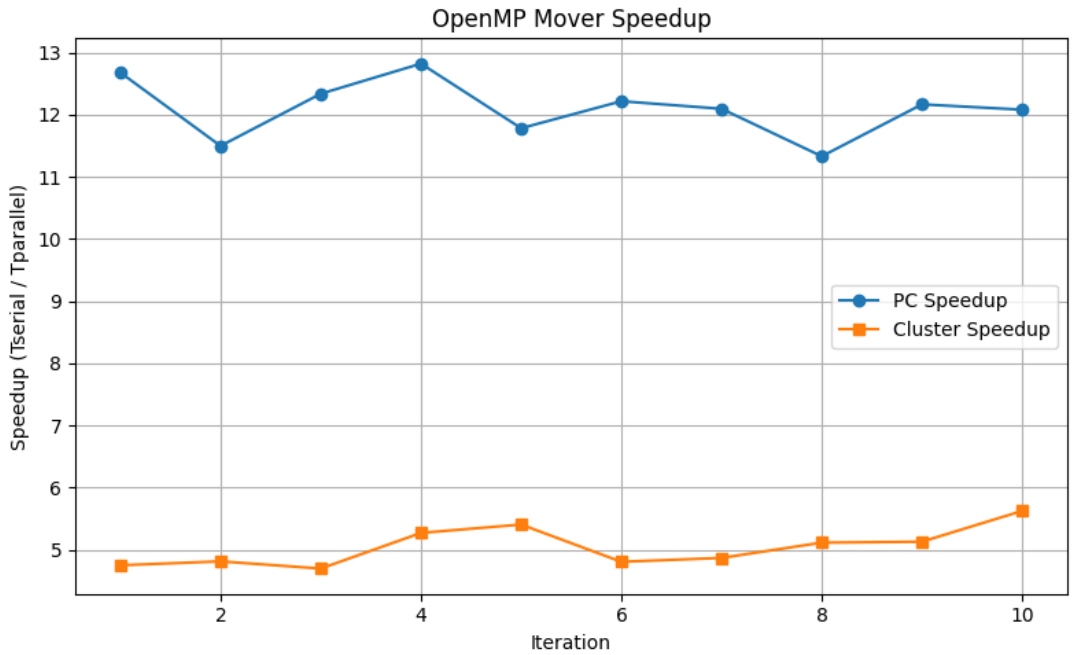

Figure 9: Speedup achieved by OpenMP parallelization of the Mover operation.

4.4 Observation

The results show that the Mover operation benefits significantly from parallelization because each particle movement is independent and can be processed concurrently.

The Lab PC shows higher speedup compared to the HPC cluster. This may be due to differences in processor architecture, memory access patterns, and thread scheduling overhead in the cluster environment.

4.5 Conclusion

Parallelizing the Mover operation using OpenMP significantly reduces execution time. The experiment demonstrates that particle movement operations are highly suitable for shared-memory parallelization and can achieve substantial performance improvements.